LABORATORIO 12

COMUNICACIÓN ENTRE SERVICIOS CON SPRING BOOT

Desarrollo de Aplicaciones Web · Sección C24-D
Estudiante: Jesús Castillo Sumire

Objetivo

Implementé una arquitectura de microservicios para administrar categorías, clientes, productos y pedidos. Eureka registra los servicios, y OpenFeign permite consultar datos remotos sin compartir tablas ni bases de datos.

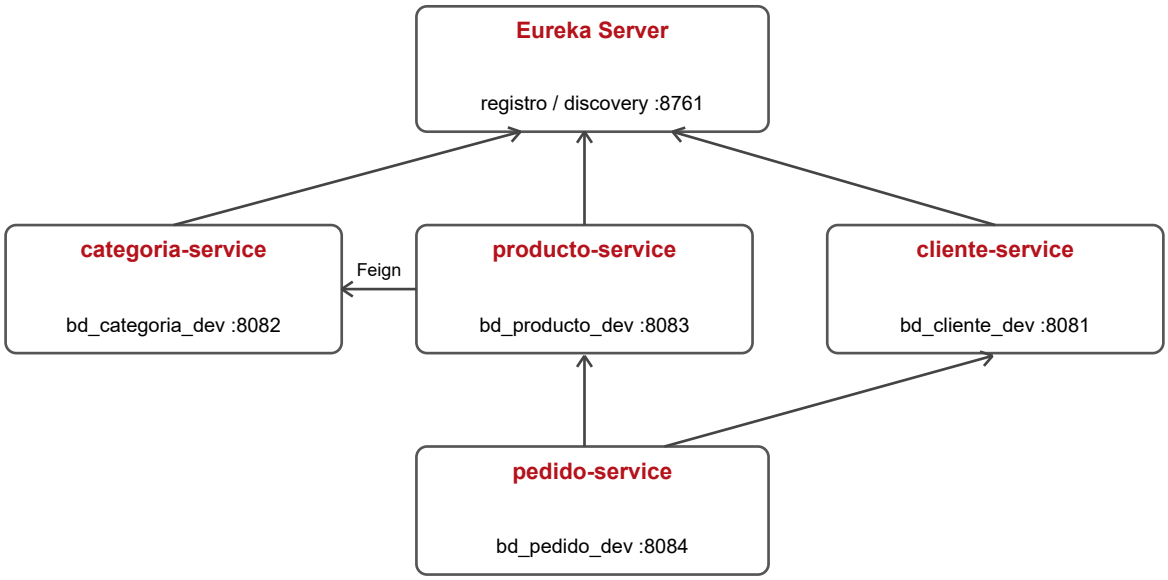
Cómo se trabajó

Organicé cinco aplicaciones Spring Boot independientes. Configuré puertos y bases de datos propios, preparé perfiles dev/prod y definí endpoints REST. El servicio de pedidos conserva identificadores externos; al crear o consultar un pedido obtiene los detalles actuales del cliente y del producto por HTTP.

Herramientas y configuración

Java 21, Spring Boot 4.1.0, Spring Cloud 2025.1.2, Spring Data JPA/Hibernate, OpenFeign, Eureka, Maven, MySQL de XAMPP y Postman. La conexión local apunta a MySQL por el puerto 3306.

Arquitectura implementada



Las llamadas entre servicios viajan por HTTP/Feign; cada servicio conserva su propia base de datos.

Figura 1. Esquema del registro y la comunicación entre los cinco servicios. El dibujo representa la arquitectura del código; no es una captura de una ejecución en vivo.

1. Descubrimiento de servicios

Eureka se ejecuta en el puerto 8761 y mantiene el registro central. Los microservicios se anuncian con su nombre lógico, por lo que Feign los encuentra sin fijar sus direcciones IP.

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

La anotación @EnableEurekaServer activa el registro. La clase principal inicia el contexto de Spring Boot.

```
server:
  port: ${SERVER_PORT:8761}
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

El servidor usa 8761 por defecto y no se registra a sí mismo como cliente.

2. Comunicación por OpenFeign

Producto consulta categoría; pedido consulta cliente y producto. Las interfaces Feign describen las rutas REST remotas y Eureka resuelve el nombre del servicio.

```
@FeignClient(name = "categoria-service")
public interface CategoriaClient {
    @GetMapping("/api/categorias/{id}")
    CategoriaResponse obtenerCategoria(@PathVariable("id") Long id);
}
```

La interfaz permite invocar el endpoint remoto como una llamada Java. El DTO mantiene el contrato entre servicios sin acoplar sus entidades JPA.

```
@FeignClient(name = "cliente-service")
public interface ClienteClient {
    @GetMapping("/api/clientes/{id}")
    ClienteResponse obtenerCliente(@PathVariable("id") Long id);
}
```

Existe una interfaz equivalente para producto-service. En pedido, ambas se usan para validar referencias y formar la respuesta agregada.

3. Reglas del pedido y respuesta agregada

El servicio valida el cliente y cada producto consultando los servicios dueños de esos datos. Calcula el total con el precio remoto y persiste solo el identificador del cliente y los identificadores/cantidades de los productos.

```
private Pedido construirPedido(PedidoRequest request) {
    Pedido pedido = new Pedido();
    pedido.setFecha(request.fecha());
    pedido.setIdCliente(request.idCliente());
    pedido.setEstado("PENDIENTE");
    BigDecimal total = BigDecimal.ZERO;
    for (PedidoRequest.DetalleRequest linea : request.detalles()) {
        ProductoResponse producto = validarProducto(linea.idProducto());
        DetallePedido detalle = new DetallePedido();
        detalle.setIdProducto(producto.id());
        detalle.setCantidad(linea.cantidad());
        detalle.setPedido(pedido);
        pedido.getDetalles().add(detalle);
        total = total.add(producto.precio().multiply(
                BigDecimal.valueOf(linea.cantidad())));
    }
    pedido.setTotal(total);
    return pedido;
}
```

Este bloque comprueba que los productos existan, arma los detalles y calcula el importe. No crea una relación JPA entre bases diferentes.

```
private PedidoResponse armarRespuesta(Pedido pedido) {
    ClienteResponse cliente = llamarCliente(pedido.getIdCliente());
    List<PedidoResponse.DetalleResponse> detalles = pedido.getDetalles().stream()
        .map(d -> new PedidoResponse.DetalleResponse(
            d.getId(), d.getCantidad(), llamarProducto(d.getIdProducto())))
        .toList();
    return new PedidoResponse(pedido.getId(), pedido.getFecha(),
        pedido.getEstado(), pedido.getTotal(), cliente, detalles);
}
```

Al consultar un pedido, la respuesta incluye datos completos obtenidos en tiempo de consulta. Los errores remotos se traducen a respuestas HTTP apropiadas: referencia inexistente como 400 y servicio no disponible como 503.

4. Persistencia independiente

El script SQL prepara una base por servicio. Hibernate crea las tablas al iniciar cada aplicación con su perfil dev. Los servicios no realizan consultas directas a las bases de los demás.

```
CREATE DATABASE IF NOT EXISTS bd_categoria_dev
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE IF NOT EXISTS bd_cliente_dev
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE IF NOT EXISTS bd_producto_dev
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE IF NOT EXISTS bd_pedido_dev
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Importar microservicios/database/create-databases.sql en phpMyAdmin antes del inicio de las aplicaciones. Debe estar activo MySQL de XAMPP en 3306.

5. Rutas principales y pruebas preparadas

Servicio	Puerto	Endpoints principales
Categoría	8082	GET, POST /api/categorias; GET, PUT, DELETE /api/categorias/{id}
Cliente	8081	GET, POST /api/clientes; GET, PUT, DELETE /api/clientes/{id}
Producto	8083	GET, POST /api/productos; GET, PUT, DELETE /api/productos/{id}
Pedido	8084	GET, POST /api/pedidos; GET, PUT /api/pedidos/{id}; PATCH /api/pedidos/{id}/estado

La colección microservicios/postman/Laboratorio-12.postman_collection.json contiene solicitudes en secuencia para validar Eureka, altas/listas, pedido agregado y rechazos de datos inválidos. Se preparó para ejecutarla cuando los servicios estén activos.

Verificación realizada

La compilación Maven de los cinco módulos terminó correctamente (BUILD SUCCESS). También se revisaron las configuraciones, las rutas y el formato JSON de la colección. No se ejecutaron llamadas HTTP reales: el puerto 3306 de MySQL no respondió en esta máquina. Por ello, este informe no afirma que Eureka haya registrado servicios ni que Postman haya aprobado solicitudes.

Pasos para ejecutar en VS Code

1. Iniciar MySQL desde XAMPP y confirmar el puerto 3306.
2. Importar el SQL de las cuatro bases de datos en phpMyAdmin.
3. Abrir una terminal en la carpeta microservicios.
4. Ejecutar Eureka y luego categoría, cliente, producto y pedido, cada uno en una terminal distinta.
5. Confirmar los cuatro servicios en http://localhost:8761.
6. Importar y correr la colección de Postman en el orden indicado.

Conclusiones

1. Comprendí que Eureka simplifica la localización de servicios porque cada aplicación se registra con un nombre lógico.

2. OpenFeign permite mantener las llamadas REST entre servicios organizadas en interfaces y DTOs.

3. Separar las bases de datos evita que un microservicio dependa de las tablas internas de otro.

4. Al crear un pedido, validar referencias por HTTP ayuda a detectar IDs inválidos y servicios fuera de línea.

5. La compilación confirmó que los módulos están integrados a nivel de código; aún debo iniciar XAMPP y ejecutar Postman para completar la comprobación funcional.

Alcance de las evidencias

El diagrama de este informe es una ilustración vectorial de la arquitectura. No se insertan imágenes de VS Code, XAMPP ni Postman porque no se realizaron capturas auténticas de esas aplicaciones durante una ejecución integrada. La prueba funcional queda pendiente de MySQL activo y de correr la colección.